

ODOO DEVELOPER PROGRAM

Automation and Reports (QWeb)

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

- 1 Introduction
- 2 Scheduled Actions (`ir.cron`)
- 3 QWeb Templates
- 4 Report Actions
- 5 Summary

Introduction

WHAT WILL WE LEARN?

- ▶ **Scheduled Actions** (`ir.cron`) – automatic background jobs
- ▶ **QWeb templates** – XML-based templating for reports and HTML
- ▶ **Report actions** – printable PDF/HTML reports from Odoo

WHY AUTOMATION AND REPORTS MATTER

- ▶ Not every process should wait for a user click.
- ▶ Cron jobs handle reminders, cleanups, syncs, and nightly computations.
- ▶ Reports turn records into professional PDFs for business use.
- ▶ QWeb is the engine behind Odoo report layouts.

WHERE THESE PIECES FIT

- 1 **Cron** decides *when* automated Python code runs.
- 2 **QWeb** decides *how* HTML/report content is rendered.
- 3 **Report action** decides *which* records and template produce a printable document.

Scheduled Actions (`ir.cron`)

WHAT IS `ir.cron`?

- ▶ **`ir.cron`** is Odoo's scheduled action (cron job) model.
- ▶ It runs a Python method automatically on a schedule.
- ▶ Typical uses:
 - send reminder emails
 - archive old records
 - compute nightly statistics
 - sync data with external systems

Core Idea

A cron points to a model method and a timing interval.

DEFINING A CRON IN XML

```
1 <record id="ir_cron_student_reminder" model="ir.cron">
2   <field name="name">Student: Send Attendance Reminder</field>
3   <field name="model_id" ref="model_student_student"/>
4   <field name="state">code</field>
5   <field name="code">model._cron_send_attendance_reminder()</field>
6   <field name="interval_number">1</field>
7   <field name="interval_type">days</field>
8   <field name="numbertocall">-1</field>
9   <field name="active" eval="True"/>
10 </record>
```

BREAKING DOWN CRON FIELDS

- ▶ **name**: label in Settings → Technical → Scheduled Actions
- ▶ **model_id**: model that owns the method
- ▶ **state**: usually `code`
- ▶ **code**: Python expression executed by the cron
- ▶ **interval_number** / **interval_type**: how often it runs
- ▶ **numercall**: how many times (-1 = forever)
- ▶ **active**: enable/disable the cron

INTERVAL TYPES

► Common interval_type values:

- minutes
- hours
- days
- weeks
- months

```
1 <field name="interval_number">30</field>  
2 <field name="interval_type">minutes</field>
```

► Choose intervals carefully – too frequent jobs can overload the server.

WRITING THE CRON METHOD

Method Syntax

```
1 @api.model
2 def _cron_send_attendance_reminder(self):
3     students = self.search([
4         ('active', '=', True),
5         ('email', '!=', False),
6     ])
7     template = self.env.ref(
8         'school_manager.mail_template_attendance_reminder',
9         raise_if_not_found=False,
10    )
11    if not template:
12        return False
13    for student in students:
14        template.send_mail(student.id, force_send=False)
15    return True
```

IMPORTANT CRON METHOD RULES

- ▶ Use `@api.model` for cron entry methods.
- ▶ Keep the method name private-style: `_cron_...`
- ▶ Make the job **idempotent** when possible (safe if run twice).
- ▶ Avoid huge work in one transaction; batch when needed.
- ▶ Log failures clearly.

```
1 @api.model
2 def _cron_archive_old_logs(self):
3     old = self.env['student.log'].search([
4         ('create_date', '<', fields.Datetime.subtract(
5             fields.Datetime.now(), days=90)),
6     ])
7     old.unlink()
```

MANUAL TESTING OF A CRON

```
1 # Odoo shell
2 env[ 'student.student' ]. _cron_send_attendance_reminder ()
3
4 # Or open Settings > Technical > Automation > Scheduled Actions
5 # and click Run Manually
```

- ▶ Always test the method manually before relying on the schedule.
- ▶ Check server logs after the first automatic runs.

USER, COMPANY, AND RIGHTS IN CRONS

- ▶ Cron jobs usually run as the OdooBot / superuser environment.
- ▶ Still write safe domains and business checks.
- ▶ If you need a specific company:

```
1 @api.model
2 def _cron_company_cleanup(self):
3     for company in self.env['res.company'].search([]):
4         students = self.with_company(company).search([
5             ('company_id', '=', company.id),
6             ('active', '=', False),
7         ])
8     # company-specific processing
```

CRON BEST PRACTICES

- ▶ One cron = one clear responsibility.
- ▶ Prefer nightly/hourly jobs over very short loops.
- ▶ Use `numercall = -1` for recurring jobs.
- ▶ Keep XML cron definitions in `data/` with `noupdate` when users may customize schedules.
- ▶ Monitor runtime and failure logs in production.

QWeb Templates

WHAT IS QWEB?

- ▶ **QWeb** is Odoo's XML templating engine.
- ▶ It renders HTML from XML templates + Python values.
- ▶ Used for:
 - PDF/HTML reports
 - website snippets and pages
 - some portal and mail rendering flows

Core Idea

Template + values $\Rightarrow$ rendered HTML.

MINIMAL QWEB REPORT TEMPLATE

```
1 <template id="report_student_card">
2   <t t-call="web.html_container">
3     <t t-foreach="docs" t-as="o">
4       <t t-call="web.external_layout">
5         <div class="page">
6           <h2>Student Card</h2>
7           <p><strong>Name:</strong> <t t-esc="o.name"/></p>
8           <p><strong>Code:</strong> <t t-esc="o.code"/></p>
9           <p><strong>Classroom:</strong>
10            <t t-esc="o.classroom_id.display_name"/>
11          </p>
12        </div>
13      </t>
14    </t>
15  </t>
16 </template>
```

BREAKING DOWN KEY QWEB DIRECTIVES

- ▶ **t-esc**: escape and print a value (safe HTML escaping)
- ▶ **t-out**: modern output directive (preferred in newer code)
- ▶ **t-raw**: print raw HTML (use carefully)
- ▶ **t-if** / **t-elif** / **t-else**: conditions
- ▶ **t-foreach** / **t-as**: loops
- ▶ **t-call**: include/call another template
- ▶ **t-set** / **t-value**: create variables

CONDITIONS AND LOOPS

```
1 <t t-if="o.email">
2   <p>Email: <t t-esc="o.email"/></p>
3 </t>
4 <t t-else="">
5   <p>No email on file</p>
6 </t>
7
8 <ul>
9   <t t-foreach="o.attendance_ids" t-as="line">
10    <li>
11      <t t-esc="line.date"/> -
12      <t t-esc="line.status"/>
13    </li>
14  </t>
15 </ul>
```

VARIABLES WITH T-SET

```
1 <t t-set="full_name" t-value="o.name"/>
2 <t t-set="is_adult" t-value="o.age >= 18"/>
3
4 <p>Student: <t t-esc="full_name"/></p>
5 <p t-if="is_adult">This student is an adult.</p>
```

- ▶ Keep expressions simple and readable.
- ▶ Heavy business logic belongs in Python, not deep inside templates.

LAYOUT HELPERS

► Common report wrappers:

- `web.html.container`
- `web.external_layout`
- `web.internal_layout`

```
1 <t t-call="web.html.container">
2   <t t-foreach="docs" t-as="o">
3     <t t-call="web.external_layout">
4       <div class="page">
5         <!-- report body -->
6       </div>
7     </t>
8   </t>
9 </t>
```

► `external_layout` adds company header/footer.

- ▶ In reports, `docs` is the recordset being printed.
- ▶ Each record is usually looped as `o`.

```
1 <t t-foreach="docs" t-as="o">  
2   <p><t t-esc="o.name"/></p>  
3 </t>
```

- ▶ Extra values can come from a custom report parser / abstract model.

STYLING QWEB REPORTS

```
1 <div class="page">
2   <h2 class="text-center">Student Report</h2>
3   <table class="table table-sm o_main_table">
4     <thead>
5       <tr>
6         <th>Name</th>
7         <th>Age</th>
8         <th>Classroom</th>
9       </tr>
10    </thead>
11    <tbody>
12      <tr>
13        <td><t t-esc="o.name"/></td>
14        <td><t t-esc="o.age"/></td>
15        <td><t t-esc="o.classroom_id.display_name"/></td>
16      </tr>
17    </tbody>
18  </table>
19 </div>
```

- ▶ Keep templates readable and modular with `t-call`.
- ▶ Prefer `t-esc/t-out` over `t-raw`.
- ▶ Put complex calculations in Python methods/fields.
- ▶ Reuse company layout templates for consistent branding.
- ▶ Test with records that have empty related fields.

Report Actions

WHAT IS A REPORT ACTION?

- ▶ A **report action** connects:
 - 1 a model
 - 2 a QWeb template
 - 3 a printable output (usually PDF)
- ▶ Stored mainly as `ir.actions.report`.

Core Idea

Select records → render QWeb → download/print PDF.

DEFINING A REPORT ACTION

```
1 <record id="action_report_student_card" model="ir.actions.report">
2   <field name="name">Student Card</field>
3   <field name="model">student.student</field>
4   <field name="report_type">qweb-pdf</field>
5   <field name="report_name">school_manager.report_student_card</field>
6   <field name="report_file">school_manager.report_student_card</field>
7   <field name="print_report_name">'Student Card - %s' % (object.name)</field>
8   <field name="binding_model_id" ref="model.student_student"/>
9   <field name="binding_type">report</field>
10 </record>
```

BREAKING DOWN REPORT ACTION FIELDS

- ▶ **name:** label in Print menu
- ▶ **model:** source model
- ▶ **report_type:** qweb-pdf or qweb-html
- ▶ **report_name / report_file:** QWeb template XML ID path
- ▶ **print_report_name:** downloaded file name expression
- ▶ **binding_model_id:** shows report in the model's Print/Action menu

FULL REPORT FILE PATTERN

```
1 <!-- 1) Report action -->
2 <record id="action_report_student_card" model="ir.actions.report">
3   <field name="name">Student Card</field>
4   <field name="model">student.student</field>
5   <field name="report_type">qweb-pdf</field>
6   <field name="report_name">school_manager.report_student_card</field>
7   <field name="report_file">school_manager.report_student_card</field>
8   <field name="binding_model_id" ref="model_student_student"/>
9 </record>
10
11 <!-- 2) QWeb template -->
12 <template id="report_student_card">
13   <t t-call="web.html_container">
14     <t t-foreach="docs" t-as="o">
15       <t t-call="web.external_layout">
16         <div class="page">
17           <h2><t t-esc="o.name"/></h2>
18         </div>
19       </t>
```

CALLING A REPORT FROM PYTHON

```
1 def action_print_student_card(self):  
2     self.ensure_one()  
3     return self.env.ref(  
4         'school_manager.action_report_student_card'  
5     ).report_action(self)
```

- ▶ `report_action(self)` opens/downloads the report for the recordset.
- ▶ Useful on form buttons: “Print Student Card”.

BUTTON TO PRINT A REPORT

```
1 <header>
2   <button name="action_print_student_card"
3     type="object"
4     string="Print Card"
5     class="btn-secondary"/>
6 </header>
```

- Users can also use the Print menu when `binding_model_id` is set.

CUSTOM REPORT PARSER (OPTIONAL)

```
1 class ReportStudentCard(models.AbstractModel):
2     _name = 'report.school_manager.report_student_card'
3     _description = 'Student Card Report'
4
5     @api.model
6     def _get_report_values(self, docids, data=None):
7         docs = self.env['student.student'].browse(docids)
8         return {
9             'doc_ids': docids,
10            'doc_model': 'student.student',
11            'docs': docs,
12            'company': self.env.company,
13        }
```

- Use this when the template needs extra computed values.

PAPER FORMAT

```
1 <record id="paperformat_student_card" model="report.paperformat">
2   <field name="name">Student Card Paper</field>
3   <field name="format">A4</field>
4   <field name="orientation">Portrait</field>
5   <field name="margin_top">40</field>
6   <field name="margin_bottom">20</field>
7 </record>
8
9 <record id="action_report_student_card" model="ir.actions.report">
10   <field name="paperformat_id" ref="paperformat_student_card"/>
11 </record>
```

REPORT ACTION BEST PRACTICES

- ▶ Keep one report action per business document type.
- ▶ Use clear print names for downloaded files.
- ▶ Bind reports only to models where they make sense.
- ▶ Test empty fields, long names, and multi-record printing.
- ▶ Put report XML in `report/` and list it in the manifest.

Summary

AUTOMATION AND REPORTS SUMMARY

- ▶ **ir.cron** runs model methods on a schedule.
- ▶ **QWeb** renders XML templates into HTML for reports and pages.
- ▶ **Report actions** bind a model + QWeb template into printable PDF/HTML output.

PRACTICAL CHECKLIST

- ▶ Need nightly automation? → `ir.cron + @api.model` method
- ▶ Need HTML structure for documents? → `QWeb` template
- ▶ Need Print menu / PDF button? → `ir.actions.report`
- ▶ Need extra report values? → `report.<module>.<template>` abstract model

SUGGESTED MODULE LAYOUT

```
1 school_manager/  
2   data/  
3     ir_cron_data.xml  
4   report/  
5     student_card_report.xml  
6   models/  
7     student_student.py  
8     report_student_card.py  
9   __manifest__.py
```

END-TO-END FLOW

- 1 Create business records (students, invoices, ...)
- 2 Automate processes with scheduled actions
- 3 Design QWeb layouts for documents
- 4 Expose report actions on forms and print menus
- 5 Test cron manually and print sample PDFs